```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score
```

```
df1 = pd.read_csv("/content/Housing (1).csv")
df1.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnis
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	yes	
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	no	
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	yes	se
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	yes	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	no	

Next steps: Generate code with df1 | New interactive sheet

```
df2 = df1.iloc[:,0:2]
df2.head()
```

	price	area
0	13300000	7420
1	12250000	8960
2	12250000	9960
3	12215000	7500
4	11410000	7420

Next steps: Generate code with df2 | New interactive sheet

```
df2.sample(5)
```

	price	area
290	4200000	2610
539	1855000	2990
452	3150000	9000
358	3745000	3480
530	2240000	1950

```
df2.describe()
```

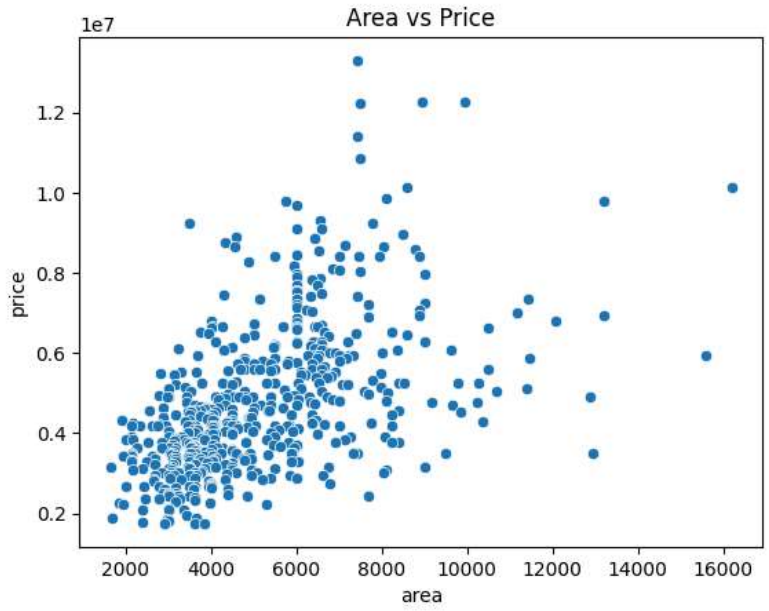
	price	area
count	5.450000e+02	545.000000
mean	4.766729e+06	5150.541284
std	1.870440e+06	2170.141023
min	1.750000e+06	1650.000000
25%	3.430000e+06	3600.000000
50%	4.340000e+06	4600.000000
75%	5.740000e+06	6360.000000
max	1.330000e+07	16200.000000

```
df2.isnull().sum()
```

	0
price	0
area	0

dtype: int64

```
# visualization

sns.scatterplot(x=df2["area"],y=df2["price"])
plt.title("Area vs Price")
plt.show()
```




```
X = df2.iloc[:,1]
y = df2.iloc[:,0]
```

```
X.head()
```

	area
0	7420
1	8960
2	9960
3	7500
4	7420

dtype: int64

```
y.head()
```

	price
0	13300000
1	12250000
2	12250000
3	12215000
4	11410000

dtype: int64

```
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.2,random_state=42
```

```
)
```

```
X_train.head()
```

	area
46	6000
93	7200
335	3816
412	2610
471	3750

dtype: int64

```
X_test
```

	area
316	5900
77	6500
360	4040
90	5000
493	3960
...	...
15	6000
357	6930
39	6000
54	6000
155	6100

109 rows × 1 columns

dtype: int64

```
y_test
```

	price
316	4060000
77	6650000
360	3710000
90	6440000
493	2800000
...	...
15	9100000
357	3773000
39	7910000
54	7350000
155	5530000

109 rows × 1 columns

dtype: int64

```
X_train = X_train.values.reshape(-1,1)
```

```
model = LinearRegression()
model.fit(X_train,y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
y_pred = model.predict(X_test.values.reshape(-1,1))
```

```
y_pred
```

```
array([5024060.33139816, 5279498.23656143, 4232202.82539203,
       4640903.47365326, 4198144.43803692, 5373158.80178796,
       6139472.51727777, 4636646.17523387, 3891618.951841  ,
       3661724.83719406, 6165016.3077941 , 4187075.46214652,
       4095969.27597162, 3832016.77396957, 4202401.73645631,
       4057653.59019713, 3363713.94783691, 5066633.31559204,
       5002773.83930122, 5066633.31559204, 4649418.07049203,
       5417860.43519154, 4065742.45719396, 4130027.66332672,
       6024525.4599543 , 6752523.48966962, 3827759.47555018,
       3789443.78977569, 8131888.17755128, 3789443.78977569,
       4215173.63171447, 3840531.37080835, 5066633.31559204,
       5328457.16838439, 4545114.25921703, 4470611.53687774,
       4490195.10960693, 3866075.16132467, 3993794.11390631,
       3698763.33344273, 5909578.40263083, 4057653.59019713,
       5245439.84920633, 4300319.60010223, 5385930.69704613,
       5017674.38376908, 5066633.31559204, 4487640.7305553 ,
       5939379.49156655, 3789443.78977569, 5694584.83245175,
       3789443.78977569, 5820175.13582369, 4428038.55268387,
       4061910.88861651, 3751128.1040012 , 4960200.85510734,
       3534005.88461242, 5258211.74446449, 4130027.66332672,
       3866075.16132467, 3421187.47649864, 5253954.4460451 ,
       4623874.27997571, 4794166.21675122, 4112998.46964917,
       5322071.22075531, 3789443.78977569, 6088384.93624512,
       4938914.36301041, 3725584.31348487, 5219896.05869   ,
       6088384.93624512, 6982417.60431656, 3654913.15972304,
       4325863.39061856, 4057653.59019713, 4562143.45289458,
       5066633.31559204, 4768622.42623489, 5219896.05869   ,
       4249232.01906958, 4853768.39462265, 5351872.30969102,
       3235994.99525527, 5049604.12191449, 3704297.82138794,
       3981022.21864814, 5671169.69114511, 5066633.31559204,
       3323269.61285272, 5939379.49156655, 4832481.90252571,
       3963993.02497059, 6165016.3077941 , 4087454.67913284,
       4993407.78277857, 4981487.34720428, 4853768.39462265,
       5322071.22075531, 3891618.951841  , 4470611.53687774,
       3776671.89451753, 3917162.74235733, 5066633.31559204,
       5462562.06859511, 5066633.31559204, 5066633.31559204,
       5109206.29978592])
```

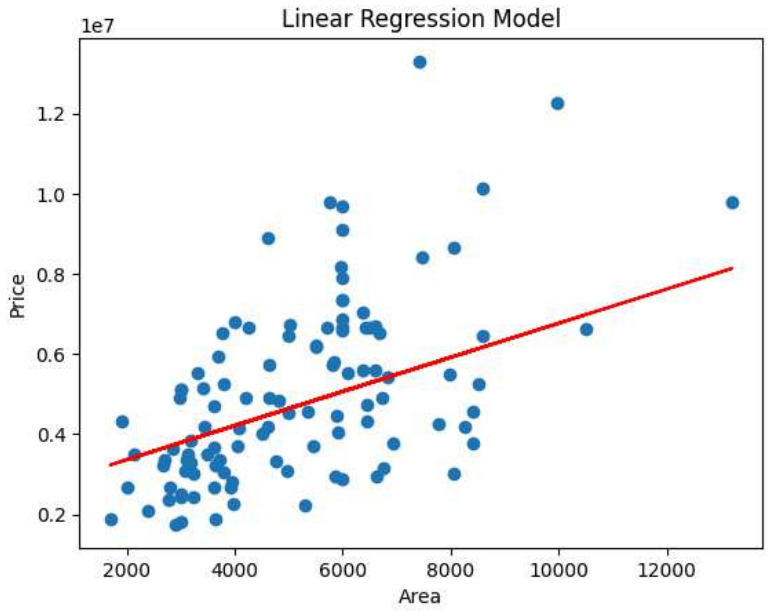
```
model.predict([[5000]])
```

```
array([4640903.47365326])
```

```
mse = mean_squared_error(y_test,y_pred)

r2 = r2_score(y_test,y_pred)

print("MSE:",mse)

print("R2 Score:",r2)
```

```
MSE: 3675286604768.185
R2 Score: 0.27287851871974644
```

```
plt.scatter(X_test,y_test)

plt.plot(X_test,y_pred,color="red")

plt.xlabel("Area")

plt.ylabel("Price")

plt.title("Linear Regression Model")

plt.show()
```




```
import pickle

pickle.dump(model,open("model.pkl","wb"))
```

```
model = pickle.load(open("model.pkl","rb"))

area = float(input("Enter house area: "))

prediction = model.predict([[area]])

print("Predicted price:",prediction)
```

```
Enter house area: 6900
Predicted price: [5449790.17333695]
```

```
print("Slope:",model.coef_)
print("Intercept:",model.intercept_)
```

```
Slope: [425.72984194]
Intercept: 2512254.2639593435
```

SEcond method

```
class LinearRegression:

    def __init__(self):
        self.m = None
        self.b = None

    def fit(self,X_train,y_train):

        num = 0
        den = 0

        for i in range(X_train.shape[0]):

            num = num + ((X_train[i] - X_train.mean())*(y_train[i] - y_train.mean()))
            den = den + ((X_train[i] - X_train.mean()))*((X_train[i] - X_train.mean()))

        self.m = num/den
        self.b = y_train.mean() - (self.m * X_train.mean())
        print(self.m)
        print(self.b)

    def predict(self,X_test):
        print(X_test)
        return self.m * X_test + self.b
```

```
df2 = df1.iloc[:,0:2]
df2.head()
```

	price	area
0	13300000	7420
1	12250000	8960
2	12250000	9960
3	12215000	7500
4	11410000	7420

Next steps: Generate code with df2 | New interactive sheet

```
X = df1.iloc[:,1].values
y = df1.iloc[:,0].values
```

```
X
```

```
array([ 8875,  7950,  5500,  7475,  7000,  4880,  5960,  6840,  7000,
        7482,  9000,  6000,  6000,  6550,  6360,  6480,  6000,  6000,
        6000,  6000,  6600,  4300,  7440,  7440,  6325,  6000,  5150,
        6000,  6000, 11440,  9000,  7680,  6000,  6000,  8880,  6240,
        6360, 11175,  8880, 13200,  7700,  6000, 12090,  4000,  6000,
        5020,  6600,  4040,  4260,  6420,  6500,  5700,  6000,  6000,
        4000, 10500,  6000,  3760,  8250,  6670,  3960,  7410,  8580,
        5000,  6750,  4800,  7200,  6000,  4100,  9000,  6400,  6600,
        6000,  6600,  5500,  5500,  6350,  5500,  4500,  5450,  6420,
        3240,  6615,  6600,  8372,  4300,  9620,  6800,  8000,  6900,
        3700,  6420,  7020,  6540,  7231,  6254,  7320,  6525, 15600,
        7160,  6500,  5500, 11460,  4800,  5828,  5200,  4800,  7000,
        6000,  5400,  4640,  5000,  6360,  5800,  6660, 10500,  4800,
        4700,  5000, 10500,  5500,  6360,  6600,  5136,  4400,  5400,
        3300,  3650,  6100,  6900,  2817,  7980,  3150,  6210,  6100,
        6600,  6825,  6710,  6450,  7800,  4600,  4260,  6540,  5500,
       10269,  8400,  5300,  3800,  9800,  8520,  6050,  7085,  3180,
        4500,  7200,  3410,  7980,  3000,  3000, 11410,  6100,  5720,
        3540,  7600, 10700,  6600,  4800,  8150,  4410,  7686,  2800,
        5948,  4200,  4520,  4095,  4120,  5400,  4770,  6300,  5800,
        3000,  2970,  6720,  4646, 12900,  3420,  4995,  4350,  4160,
        6040,  6862,  4815,  7000,  8100,  3420,  9166,  6321, 10240,
        6440,  5170,  6000,  3630,  9667,  5400,  4320,  3745,  4160,
        3880,  5680,  2870,  5010,  4510,  4000,  3840,  3760,  3640,
        2550,  5320,  5360,  3520,  8400,  4100,  4990,  3510,  3450,
        9860,  3520,  4510,  5885,  4000,  8250,  4040,  6360,  3162,
        3510,  3750,  3968,  4900,  2880,  4880,  4920,  4950,  3900,
        4500,  1905,  4075,  3500,  6450,  4032,  4400, 10360,  3400,
        6360,  6360,  4500,  2175,  4360,  7770,  6650,  2787,  5500,
        5040,  5850,  2610,  2953,  2747,  4410,  4000,  2325,  4600,
        3640,  5800,  7000,  4079,  3520,  2145,  4500,  8250,  3450,
        4840,  4080,  4046,  4632,  5985,  6060,  3600,  3680,  4040,
        5600,  5900,  4992,  4340,  3000,  4320,  3630,  3460,  5400,
        4500,  3460,  4100,  6480,  4500,  3960,  4050,  7260,  5500,
        3000,  3290,  3816,  8080,  2145,  3780,  3180,  5300,  3180,
        7152,  4080,  3850,  2015,  2176,  3350,  3150,  4820,  3420,
        3600,  5830,  2856,  8400,  8250,  2520,  6930,  3480,  3600,
        4040,  6020,  4050,  3584,  3120,  5450,  3630,  3630,  5640,
        3600,  4280,  3570,  3180,  3000,  3520,  5960,  4130,  2850,
        2275,  3520,  4500,  4000,  3150,  4500,  4500,  3640,  3850,
        4240,  3650,  4600,  2135,  3036,  3990,  7424,  3480,  3600,
        3640,  5900,  3120,  7350,  3512,  9500,  5880, 12944,  4900,
        3060,  5320,  2145,  4000,  3185,  3850,  2145,  2610,  1950,
        4040,  4785,  3450,  3640,  3500,  4960,  4120,  4750,  3720,
        3750,  3100,  3185,  2700,  2145,  4040,  4775,  2500,  3180,
        6060,  3480,  3792,  4040,  2145,  5880,  4500,  3930,  3640,
        4370,  2684,  4320,  3120,  3450,  3986,  3500,  4095,  1650,
        3450,  6750,  9000,  3069,  4500,  5495,  2398,  3000,  3850,
        3500,  8100,  4960,  2160,  3090,  4500,  3800,  3090,  3240,
        2835,  4600,  5076,  3750,  3630,  8050,  4352,  3000,  5850,
        4960,  3600,  3660,  3480,  2700,  3150,  6615,  3040,  3630,
        6000,  5400,  5200,  3300,  4350,  2640,  2650,  3960,  6800,
        4000,  4000,  3934,  2000,  3630,  2800,  2430,  3480,  4000,
        3185,  4000,  2910,  3600,  4400,  3600,  2880,  3180,  3000,
        4400,  3000,  3210,  3240,  3000,  3500,  4840,  7700,  3635,
        2475,  2787,  3264,  3640,  3180,  1836,  3970,  3970,  1950,
        5300,  3000,  2400,  3000,  3360,  3420,  1700,  3649,  2990,
        3000,  2400,  3620,  2910,  3850])
```

```
y
```

```
        4935000,  4907000,  4900000,  4900000,  4900000,  4900000,
        4900000,  4900000,  4900000,  4900000,  4900000,  4900000,
        4900000,  4900000,  4893000,  4893000,  4865000,  4830000,
        4830000,  4830000,  4830000,  4795000,  4795000,  4767000,
        4760000,  4760000,  4760000,  4753000,  4690000,  4690000,
        4690000,  4690000,  4690000,  4690000,  4655000,  4620000,
        4620000,  4620000,  4620000,  4620000,  4613000,  4585000,
        4585000,  4550000,  4550000,  4550000,  4550000,  4550000,
        4550000,  4550000,  4543000,  4543000,  4515000,  4515000,
        4515000,  4515000,  4480000,  4480000,  4480000,  4480000,
        4480000,  4473000,  4473000,  4473000,  4445000,  4410000,
        4410000,  4403000,  4403000,  4403000,  4382000,  4375000,
        4340000,  4340000,  4340000,  4340000,  4340000,  4319000,
        4305000,  4305000,  4277000,  4270000,  4270000,  4270000,
        4270000,  4270000,  4270000,  4235000,  4235000,  4200000,
        4200000,  4200000,  4200000,  4200000,  4200000,  4200000,
        4200000,  4200000,  4200000,  4200000,  4200000,  4200000,
        4200000,  4200000,  4200000,  4200000,  4193000,  4193000,
        4165000,  4165000,  4165000,  4130000,  4130000,  4123000,
        4098500,  4095000,  4095000,  4095000,  4060000,  4060000,
        4060000,  4060000,  4060000,  4025000,  4025000,  4025000,
        4007500,  4007500,  3990000,  3990000,  3990000,  3990000,
        3990000,  3920000,  3920000,  3920000,  3920000,  3920000,
        3920000,  3920000,  3885000,  3885000,  3850000,  3850000,
        3850000,  3850000,  3850000,  3850000,  3850000,  3836000,
        3815000,  3780000,  3780000,  3780000,  3780000,  3780000,
        3780000,  3773000,  3773000,  3773000,  3745000,  3710000,
        3710000,  3710000,  3710000,  3710000,  3703000,  3703000,
        3675000,  3675000,  3675000,  3675000,  3640000,  3640000,
        3640000,  3640000,  3640000,  3640000,  3640000,  3640000,
        3640000,  3633000,  3605000,  3605000,  3570000,  3570000,
        3570000,  3570000,  3535000,  3500000,  3500000,  3500000,
        3500000,  3500000,  3500000,  3500000,  3500000,  3500000,
        3500000,  3500000,  3500000,  3500000,  3500000,  3500000,
        3500000,  3500000,  3493000,  3465000,  3465000,  3465000,
        3430000,  3430000,  3430000,  3430000,  3430000,  3430000,
        3423000,  3395000,  3395000,  3395000,  3360000,  3360000,
        3360000,  3360000,  3360000,  3360000,  3360000,  3360000,
        3353000,  3332000,  3325000,  3325000,  3290000,  3290000,
        3290000,  3290000,  3290000,  3290000,  3290000,  3290000,
        3255000,  3255000,  3234000,  3220000,  3220000,  3220000,
        3220000,  3150000,  3150000,  3150000,  3150000,  3150000,
        3150000,  3150000,  3150000,  3150000,  3143000,  3129000,
        3118850,  3115000,  3115000,  3115000,  3087000,  3080000,
        3080000,  3080000,  3080000,  3045000,  3010000,  3010000,
        3010000,  3010000,  3010000,  3010000,  3010000,  3003000,
        2975000,  2961000,  2940000,  2940000,  2940000,  2940000,
        2940000,  2940000,  2940000,  2940000,  2870000,  2870000,
        2870000,  2870000,  2852500,  2835000,  2835000,  2835000,
        2800000,  2800000,  2730000,  2730000,  2695000,  2660000,
        2660000,  2660000,  2660000,  2660000,  2660000,  2660000,
        2653000,  2653000,  2604000,  2590000,  2590000,  2590000,
        2520000,  2520000,  2520000,  2485000,  2485000,  2450000,
        2450000,  2450000,  2450000,  2450000,  2450000,  2408000,
        2380000,  2380000,  2380000,  2345000,  2310000,  2275000,
        2275000,  2275000,  2240000,  2233000,  2135000,  2100000,
        2100000,  2100000,  1960000,  1890000,  1890000,  1855000,
        1820000,  1767150,  1750000,  1750000,  1750000])
```

```
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.2,random_state=42
)
```

```
X_train.shape
```

```
(436,)
```

```
lr = LinearRegression()
```

```
lr.fit(X_train,y_train)
```

```
425.7298419387829
2512254.263959343
```

```
X_test
```

```
array([ 5900,  6500,  4040,  5000,  3960,  6720,  8520,  4990,  3240,
        2700,  8580,  3934,  3720,  3100,  3970,  3630,  2000,  6000,
        5850,  6000,  5020,  6825,  3649,  3800,  8250,  9960,  3090,
        3000, 13200,  3000,  4000,  3120,  6000,  6615,  4775,  4600,
        4646,  3180,  3480,  2787,  7980,  3630,  6420,  4200,  6750,
        5885,  6000,  4640,  8050,  3000,  7475,  3000,  7770,  4500,
        3640,  2910,  5750,  2400,  6450,  3800,  3180,  2135,  6440,
```

```
        4960,  5360,  3760,  6600,  3000,  8400,  5700,  2850,  6360,
        8400, 10500,  2684,  4260,  3630,  4815,  6000,  5300,  6360,
        4080,  5500,  6670,  1700,  5960,  2800,  3450,  7420,  6000,
        1905,  8050,  5450,  3410,  8580,  3700,  5828,  5800,  5500,
        6600,  3240,  4600,  2970,  3300,  6000,  6930,  6000,  6000,
        6100])
```

```
y_test
```

```
array([ 4060000,  6650000,  3710000,  6440000,  2800000,  4900000,
        5250000,  4543000,  2450000,  3353000, 10150000,  2660000,
        3360000,  3360000,  2275000,  2660000,  2660000,  7350000,
        2940000,  2870000,  6720000,  5425000,  1890000,  5250000,
        4193000, 12250000,  3080000,  5110000,  9800000,  2520000,
        6790000,  3500000,  6650000,  2940000,  3325000,  4200000,
        4900000,  3290000,  3500000,  2380000,  5495000,  3675000,
        6650000,  4907000,  3150000,  4480000,  6580000,  5740000,
        3003000,  1820000,  8400000,  2450000,  4270000,  4007500,
        3234000,  1750000,  9800000,  2100000,  4340000,  3045000,
        3850000,  3500000,  4753000,  3080000,  4550000,  6510000,
        6685000,  5110000,  4550000,  6650000,  3640000,  5600000,
        3780000,  6615000,  3220000,  6650000,  4690000,  4830000,
        6860000,  2233000,  7035000,  4165000,  6195000,  6510000,
        1890000,  8190000,  2660000,  4193000, 13300000,  9681000,
        4340000,  8645000,  3703000,  5145000,  6440000,  5950000,
        5810000,  5740000,  6230000,  5600000,  3010000,  8890000,
        4900000,  5530000,  9100000,  3773000,  7910000,  7350000,
        5530000])
```

```
print(lr.predict(X_test[1]))
```

```
6500
5279498.2365614325
```

```
X_test
```

```
array([ 5900,  6500,  4040,  5000,  3960,  6720,  8520,  4990,  3240,
        2700,  8580,  3934,  3720,  3100,  3970,  3630,  2000,  6000,
        5850,  6000,  5020,  6825,  3649,  3800,  8250,  9960,  3090,
        3000, 13200,  3000,  4000,  3120,  6000,  6615,  4775,  4600,
        4646,  3180,  3480,  2787,  7980,  3630,  6420,  4200,  6750,
        5885,  6000,  4640,  8050,  3000,  7475,  3000,  7770,  4500,
        3640,  2910,  5750,  2400,  6450,  3800,  3180,  2135,  6440,
        4960,  5360,  3760,  6600,  3000,  8400,  5700,  2850,  6360,
        8400, 10500,  2684,  4260,  3630,  4815,  6000,  5300,  6360,
        4080,  5500,  6670,  1700,  5960,  2800,  3450,  7420,  6000,
        1905,  8050,  5450,  3410,  8580,  3700,  5828,  5800,  5500,
        6600,  3240,  4600,  2970,  3300,  6000,  6930,  6000,  6000,
        6100])
```

```
y_test
```

```
array([ 4060000,  6650000,  3710000,  6440000,  2800000,  4900000,
        5250000,  4543000,  2450000,  3353000, 10150000,  2660000,
        3360000,  3360000,  2275000,  2660000,  2660000,  7350000,
        2940000,  2870000,  6720000,  5425000,  1890000,  5250000,
        4193000, 12250000,  3080000,  5110000,  9800000,  2520000,
        6790000,  3500000,  6650000,  2940000,  3325000,  4200000,
        4900000,  3290000,  3500000,  2380000,  5495000,  3675000,
        6650000,  4907000,  3150000,  4480000,  6580000,  5740000,
        3003000,  1820000,  8400000,  2450000,  4270000,  4007500,
        3234000,  1750000,  9800000,  2100000,  4340000,  3045000,
        3850000,  3500000,  4753000,  3080000,  4550000,  6510000,
        6685000,  5110000,  4550000,  6650000,  3640000,  5600000,
        3780000,  6615000,  3220000,  6650000,  4690000,  4830000,
        6860000,  2233000,  7035000,  4165000,  6195000,  6510000,
        1890000,  8190000,  2660000,  4193000, 13300000,  9681000,
        4340000,  8645000,  3703000,  5145000,  6440000,  5950000,
        5810000,  5740000,  6230000,  5600000,  3010000,  8890000,
        4900000,  5530000,  9100000,  3773000,  7910000,  7350000,
        5530000])
```